

PIMPRI CHINCHWAD EDUCATION TRUST'S

PIMPRI CHINCHWAD COLLEGE OF ENGINEERING

NBA Accredited | NAAC Accredited with 'A' Grade | An Autonomous Institute
AICTE Approved | Permanently Affiliated to SPPU, Pune

Department of Information Technology

Academic Year: 2026-2027

Assignment No. 5: Multistage Graph (Dynamic Programming)

Name	:	**Om Rajendra Bharambe**
PRN	:	124B1F001
Branch	:	Information Technology (IT)
Semester	:	V
Batch	:	A1
Course Code	:	BIT25PC04
Course Name	:	**Design & Analysis of Algorithms Laboratory**
Guided By	:	Dr. Rajesh N. Phursule
GitHub Repository	:	om29dev/DAA_Lab_Submission
Assignment Link	:	Assignment 5 on GitHub

Assignment No. 5: Multistage Graph

Problem Statement

A logistics company, SwiftCargo, specializes in delivering packages across multiple cities. To optimize its delivery process, the company divides the transportation network into multiple stages (warehouses, transit hubs, and final delivery points). Each package must follow the most cost-efficient or fastest route from the source to the destination while passing through these predefined stages. As a logistics optimization engineer, you must:

1. Model the transportation network as a directed, weighted multistage graph with multiple intermediate stages.

2. Implement an efficient algorithm (such as Dynamic Programming or Dijkstra's Algorithm) to find the optimal delivery route.

3. Ensure that the algorithm scales for large datasets (handling thousands of cities and routes).

4. Analyze and optimize route selection based on real-time constraints, such as traffic conditions, weather delays, or fuel efficiency.

Constraints & Considerations:

- The network is structured into N stages, and every package must pass through at least one node in each stage.
- There may be multiple paths with different costs/times between stages.
- The algorithm should be flexible enough to handle real-time changes (e.g., road closures or rerouting requirements).
- The system should support batch processing for multiple delivery requests.

Tasks:

1. Model the network as a directed weighted multistage graph.

2. Use Dynamic Programming (DP) to compute optimal delivery routes.

3. Ensure scalability for thousands of cities/routes.

4. Consider real-time constraints like traffic, weather, or fuel efficiency.

Course Objective

- To know the basics of computational complexity of various algorithms.
- To select appropriate algorithm design strategies to solve real-world problems.

Course Outcome

1. Analyze the asymptotic performance of algorithms.

2. Generate optimal solutions by applying Dynamic Programming strategy.

Theory

Multistage Graph

A multistage graph is a directed graph where the vertices are partitioned into a set of disjoint stages, V1, V2, ..., Vk, such that all edges go from a vertex in one stage to a vertex in the next stage. This structure is commonly used to model dynamic programming problems.

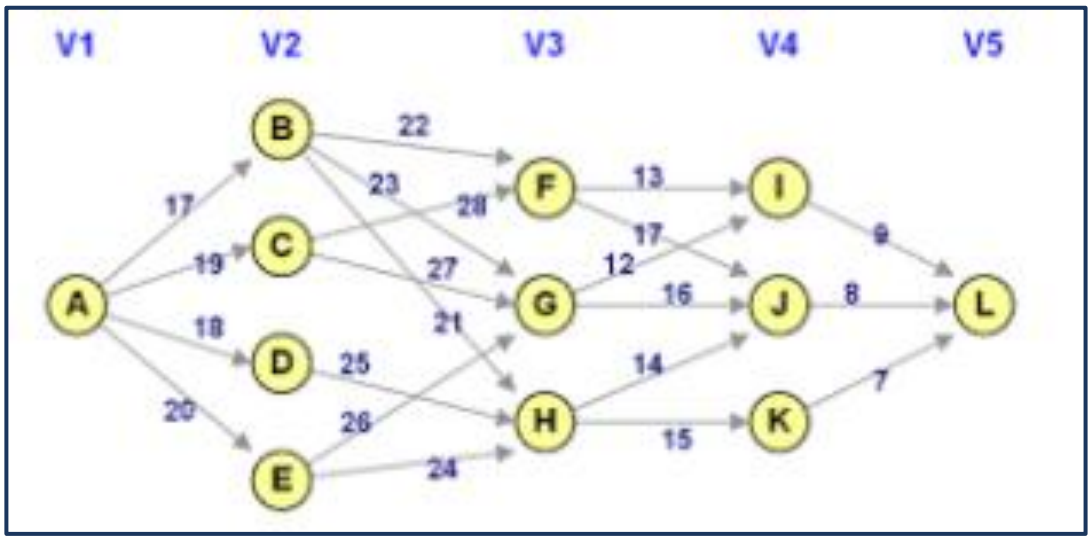


Figure 5.1: Multistage Graph Partitioned into 5 Disjoint Stages (V1 to V5)

Structure:

The graph displays vertices partitioned into five stages labeled V1 to V5:

- V1 = {A} (The source node).
- V2 = {B, C, D, E}.
- V3 = {F, G, H}.
- V4 = {I, J, K}.
- V5 = {L} (The sink node).

Edges and Costs:

The edges are directed (indicated by arrows) and only connect a vertex in stage Vi to a vertex in the immediately succeeding stage Vi+1. Each edge has an associated numerical value, which represents the cost or distance of traversing that specific edge.

Goal:

This type of graph is typically used to find the shortest path (or sometimes the longest path) from the source node (A in V1) to the sink node (L in V5).

Multistage Property:

The critical feature is that a path from A to L must pass through exactly one vertex from each intermediate stage (V2, V3, V4). This sequential, stage-by-stage progression is what defines it as a "multistage" graph.

Application (Dynamic Programming):

The shortest path problem on a multistage graph is a classic application of the principle of optimality used in dynamic programming.

- The problem can be broken down into a sequence of smaller, overlapping sub problems.
- The shortest path from A to L is found by iteratively calculating the shortest path to each vertex in a stage from all vertices in the previous stage. For example, to find the shortest path to F in V3, you consider the shortest paths to its predecessors (B, C, D in V2) and add the cost of the edge from the predecessor to F.
- The final answer is the shortest path to the sink, L.

Use Cases of Multistage Graphs in Real Life:

- **Logistics:** Routing parcels through hubs (like in SwiftCargo).
- **Speech Recognition:** Word sequence prediction (language models).
- **Manufacturing:** Sequential processes with cost/time optimization.
- **Network Routing:** Data packet transmission in layered protocols.
- **Finance:** Investment planning through time periods.

Dynamic Programming (DP)

Definition: Dynamic Programming is an optimization technique used to solve complex problems by breaking them into smaller overlapping subproblems, solving each subproblem once, and storing its result.

Key Properties:

1. **Optimal Substructure:** The solution to the main problem can be built from solutions to sub problems.
2. **Overlapping Sub problems:** The same sub problems are solved multiple times.

Why DP is Better than Recursion Here:

- Avoids recomputation of same subproblems.
- Works in bottom-up fashion (used in multistage graph).
- Saves time and memory (using memoization or tabulation).

Difference Between Greedy, DP, and Backtracking:

Strategy	Suitable When	Pros	Cons
Greedy	Local choice leads to global optimum	Fast, simple	May give suboptimal solution
DP	Optimal substructure + overlapping sub problems	Guarantees optimal, efficient	More memory use, needs planning
Backtracking	Combinatorial problems with constraints	Finds all/optimal combinations	Slow for large inputs

For this problem:

- Graph is acyclic -> no cycles.
- Problem has optimal substructure -> best path from node i depends on best paths from nodes in next stage.
- DP avoids redundant calculations by storing partial results.
- Efficient for large multistage networks (thousands of nodes).

Working of DP in a Multistage Graph:

1. Number stages 1 -> N.
2. Initialize destination cost = 0.
3. Move backward stage by stage.
4. Store next[] array to track path.
5. At the end, cost[source] gives the minimum cost, and path can be reconstructed.

Example:

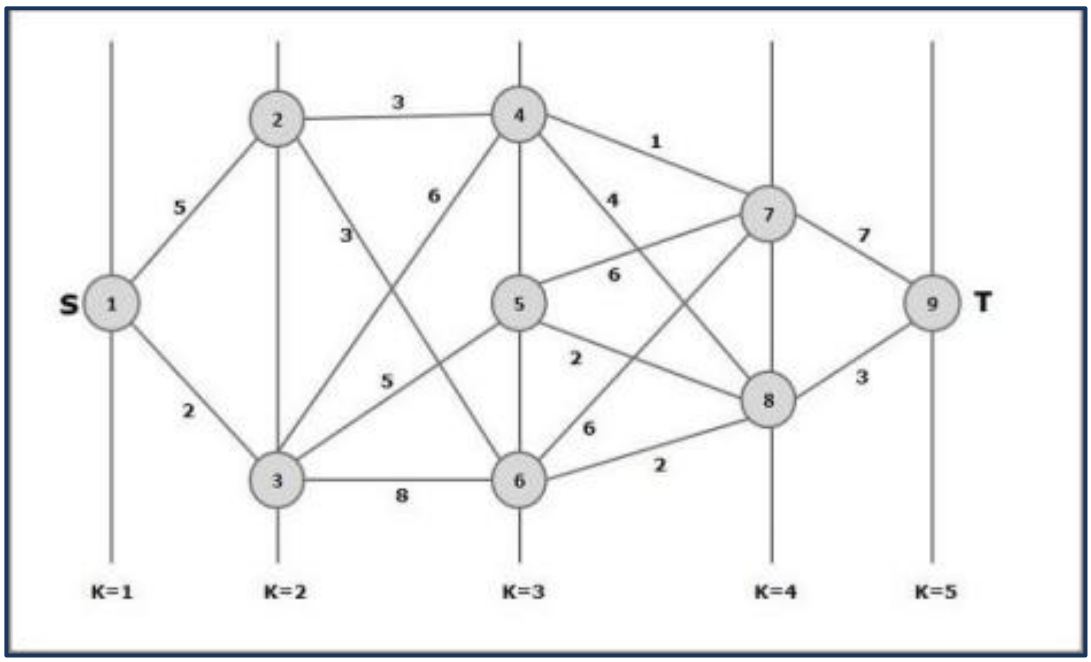


Figure 5.2: Multistage Graph Walkthrough Example (Stages K=1 to K=5)

According to the formula, we have to calculate the cost (i, j) using the following steps:

Step 1: Cost (K-2, j)

In this step, three nodes (node 4, 5, 6) are selected as j. Hence, we have three options to choose the minimum cost at this step:

- Cost(3, 4) = min {c(4, 7) + Cost(7, 9), c(4, 8) + Cost(8, 9)} = min {1 + 7, 4 + 3} = 7
- Cost(3, 5) = min {c(5, 7) + Cost(7, 9), c(5, 8) + Cost(8, 9)} = min {6 + 7, 2 + 3} = 5
- Cost(3, 6) = min {c(6, 7) + Cost(7, 9), c(6, 8) + Cost(8, 9)} = min {6 + 7, 2 + 3} = 5

Step 2: Cost (K-3, j)

Two nodes are selected as j because at stage k - 3 = 2 there are two nodes, 2 and 3. So, the value i = 2 and j = 2 and 3:

- Cost(2, 2) = min {c(2, 4) + Cost(4, 8) + Cost(8, 9), c(2, 6) + Cost(6, 8) + Cost(8, 9)} = min {3 + 7, 3 + 5} = 8
- Cost(2, 3) = min {c(3, 4) + Cost(4, 8) + Cost(8, 9), c(3, 5) + Cost(5, 8) + Cost(8, 9), c(3, 6) + Cost(6, 8) + Cost(8, 9)} = min {6 + 7, 5 + 5, 8 + 5} = 10

Step 3: Cost (K-4, j)

- Cost (1, 1) = min {c(1, 2) + Cost(2, 2), c(1, 3) + Cost(2, 3)} = min {5 + 8, 2 + 10} = 12

Hence, the path having the minimum cost is 1 -> 3 -> 5 -> 8 -> 9 with total minimum cost = 12.

Algorithm

```
Algorithm MultiStageDP(G)
1. cost[destination] = 0
2. for i = n-1 downto 1:
3.   cost[i] = ∞
4.   for each (i,j) in edges:
5.     if cost[i] > weight(i,j) + cost[j]:
6.       cost[i] = weight(i,j) + cost[j]
7.       next[i] = j
8. print cost[source], path
```

Pseudo code: Multistage Graph Shortest Path (Backward DP)

This algorithm assumes the graph G=(V,E) is partitioned into K stages, V1 (source s) to VK (sink L). N is the total number of vertices.

```
SHORTEST_PATH_MULTISTAGE(G, K, V1, ..., VK)
// Initialize Cost and Path Arrays
Create an array cost[1...N] to store the shortest distance from each node i to
the sink L.
Create an array path[1...N] to store the next node on the shortest path from i
to L.
Let L be the single node in the final stage VK.

// Base Case (Stage VK)
cost[L] <- 0

// Iterate Backward Through Stages (K-1 down to 1)
For stage k from K-1 down to 1 do:
  For each node i in Vk do:
    min_cost <- infinity
    min_next_node <- NULL
    For each successor j of i (where j in Vk+1 and (i,j) in E) do:
      current_cost <- weight(i,j) + cost[j]
      If current_cost < min_cost then:
        min_cost <- current_cost
        min_next_node <- j
      End If
    End For
    cost[i] <- min_cost
    path[i] <- min_next_node
  End For
End For
```

```
// Construct and Return the Shortest Path
Shortest Path Cost <- cost[s] // where s is the source node in V1
i <- s
While i != L do:
  Output node i
  i <- path[i]
End While
Output node L
```

Time & Space Complexity

Measure	Complexity / Description
Time	O(V + E)
Space	O(V)
Suitable for	Large multistage logistics graphs

Conclusion

1. Multistage graphs model stepwise logistics efficiently.

2. Dynamic Programming finds the optimal route by backward computation using optimal substructure.

3. Approach is scalable, flexible, and adapts to real-time constraints.

4. Ensures minimum delivery cost/time, improving SwiftCargo's operational efficiency.

Code and Output

Repository: https://github.com/om29dev/DAA_Lab_Submission

Assignment 5 Folder: https://github.com/om29dev/DAA_Lab_Submission/tree/main/Assignment 5

7.1 C++ Implementation Source Code (multistage_graph_dp.cpp):

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;
```

```
struct Edge {
    int u, v, weight;
};

int main() {
    int stages, n, m;
    cout << "Enter number of stages: ";
    if (!(cin >> stages)) return 0;

    cout << "Enter total number of vertices: ";
    cin >> n;

    cout << "Enter total number of edges: ";
    cin >> m;

    vector<vector<pair<int, int>>> adj(n + 1);
    cout << "Enter edges (u v weight):\n";
    for (int i = 0; i < m; i++) {
        int u, v, w;
        cin >> u >> v >> w;
        adj[u].push_back({v, w});
    }

    int source, sink;
    cout << "Enter source vertex: ";
    cin >> source;
    cout << "Enter sink vertex: ";
    cin >> sink;

    vector<long long> cost(n + 1, LLONG_MAX);
    vector<int> nxt(n + 1, -1);

    cost[sink] = 0;

    for (int i = sink - 1; i >= source; i--) {
        for (auto &edge : adj[i]) {
            int to = edge.first;
            int weight = edge.second;
            if (cost[to] != LLONG_MAX && cost[to] + weight < cost[i]) {
                cost[i] = cost[to] + weight;
                nxt[i] = to;
            }
        }
    }
```

```
    if (cost[source] == LLONG_MAX) {
        cout << "\nNo valid path found from source to sink.\n";
        return 0;
    }

    cout << "\nMinimum Delivery Cost: " << cost[source] << "\n";
    cout << "Optimal Delivery Route: ";
    int curr = source;
    while (curr != -1) {
        cout << curr;
        if (curr == sink) break;
        cout << " -> ";
        curr = nxt[curr];
    }
    cout << "\n";

    return 0;
}
```

7.2 Execution Output:

Compile and run multistage_graph_dp.cpp in the terminal environment:

```
om29dev@DESKTOP-MH6NVIC:~/learn$ vi multistage_graph_dp.cpp
om29dev@DESKTOP-MH6NVIC:~/learn$ g++ multistage_graph_dp.cpp -o multistage_graph_dp
om29dev@DESKTOP-MH6NVIC:~/learn$ ./multistage_graph_dp
Enter number of stages: 5
Enter total number of vertices: 9
Enter total number of edges: 15
Enter edges (u v weight):
1 2 5
1 3 2
2 4 3
2 6 3
3 4 6
3 5 5
3 6 8
4 7 1
4 8 4
5 7 6
5 8 2
6 7 6
6 8 2
7 9 7
8 9 3
Enter source vertex: 1
Enter sink vertex: 9

Minimum Delivery Cost: 12
Optimal Delivery Route: 1 -> 3 -> 5 -> 8 -> 9
om29dev@DESKTOP-MH6NVIC:~/learn$ D
```

Figure 7.1: Multistage Graph Dynamic Programming Execution Output